Part 4 Project Logbook

Project #43: Reconfigurable Neural Processing Unit (NPU) for Energy-Efficient AI at the Edge

Pratham Chhabra
pchh520@aucklanduni.ac.nz
ID: 999822254

Weekly Logbook Entry Template

Week 1 — 03/03/25 – 09/03/25:

Thursday 6th March **Time: 2PM**

Objective: Project Scope Discussion with Supervisors

- Setup a meeting time of 11:15 everyday Thursday with supervisors to discuss the status of the project.
- Need to view UoA library for research papers. (Look for IEEE) Could even search on 'IEEE xplore'
- Morteza mentioned something about "Systolic Array". – *Need to learn what that is*
- Suggested a collaborative workspace with version control. – *Morteza recommends a flash drive to be extra safe*

Week 2 — 10/03/25 – 16/03/25 XX

Tuesday 11th March 2025 **Time:** 6pm

Objective: Create a management system for P4P and start reading the modules on Canva

- Created a sharepoint – *Also shared with supervisors to make it more collaborative*
- Set up a Discord Server for efficient communication
- Chose GitHub as primary source of version control and OneDrive as secondary
- Surfed over the web and through IEEE Xplore for relevant papers.

Wednesday 12th March 2025 **Time:** 7pm

Objective: Continue researching for relevant papers and find out more about NPUs.

- "Reconfigurable" NPUs allow for dynamic adaptation to different neural networks.
- Trying to learn about the 3 different neural networks. *Not sure if it is relevant at this stage.*
- https://ieeexplore.ieee.org/document/7551407

Weekly Logbook Entry Template

Week 3 — 17/03/25 – 23/03/25:

18/03 – 19/03 **Time: All day**

Objective: Go back to the basics, gain some proper background knowledge for the project

- What are neural networks?

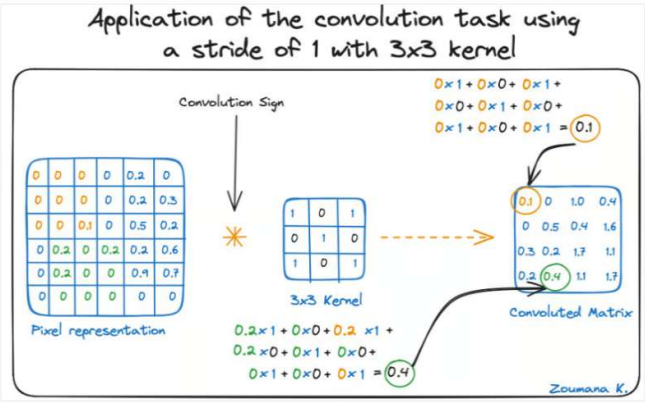
Example: recognising a number between 0-9 from a 28x28 pixel image greyscale.
Input layer: Each pixel in the image will have a value between 0 to 1 that corresponds to how bright the pixel is. This is the first layer of the network.
Hidden layers: From the input layer to a hidden layer, relevant data is identified and separated and passed onto the next hidden layers until it reaches the output layer.
Output layer: The final prediction of what it thinks the number is.

- Why are the networks layered?

The numbers can be broken down in many pieces. These pieces have a particular pattern e.g. loop, straight lines. Then, these little patterns will have smaller components. When these smaller components go through the layer, it will light up neurons that correspond to it, and as it passes through the layers, it will detect the bigger pieces until it can determine / make a prediction.

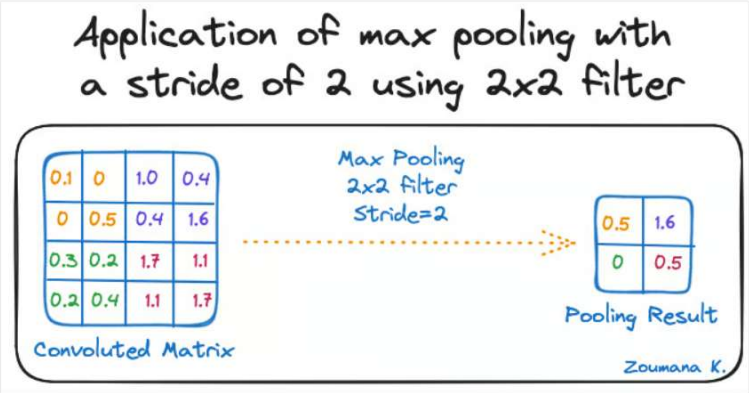
- Convolutional layer:
  - Filter: Small matrices of the same size contain a pattern that is part of the image. These matrices are compared to the entire image.

The dimension of the convoluted matrix depends on the size of the sliding window. The higher the sliding window, the smaller the dimension.



- Rectified Linear Unit (ReLU) activation function: helps learn the relationships between the features of the image

- Pooling layer:
    - Pulls the most significant features from the convoluted matrix



- Fully connected layers:
    - Output layer where all extracted features are transformed into predictions

Learning Outcomes:

- How a neural network works and its main three layers: input, hidden, and output layer
- Deep learning is based on learning patterns and requires no human intervention
- Neural networks mimic how the brain works and is trained using large batches of images
- DNN becomes “deep” because it has more than one hidden layer
- CNN uses a filter that slides around the entire images and uses that to compare
- RNN has a memory that updates as it learns through its hidden state loops

20/03/2025 **Time: 4pm-6pm**

Objective: Gain some knowledge for NPUs

Notes:

• Specialised processor to accelerate AI and ML workloads

• Tailored for complex computation

• Integrated within the main CPU but it can be discrete in some cases like data centres

• NPUs are optimised for heavy computations in deep learning like facial recognition and autonomous driving

• Avoids having to send to cloud for processing so its better privacy

Advantages:

- Parallel processor so many multiple neural network operations run concurrently

- Low precision arithmetic (usually 8 bits or lower) so computational complexity is less (cause there is less data stored) so it's more energy efficient (check https://research.ibm.com/blog/low-precision-computing)
- High-bandwidth memory: can stored and access memory quicker for tasks

How does it work?

- Purpose built for problem-solving functions that can improve overtime using different types of data and inputs
- NOT built for precise computations so that it can prioritise speed and efficiency for AI
- Has addition and multiplication modules used for AI tasks that calculate things like matrix multiplication/additional, convolution, and dot product etc
- Normal CPUs might need thousands of instructions to complete one neuron processing but NPU can do similar operation in one
- Uses synaptic weights (how strong the connection is between two nodes) for storage and computation for more efficiency

---

Week 4 — 24/03/25 – 30/03/25:

24/03/25 **Time: XX**

Title: Energy Efficiency and systolic arrays

- Because there is so many instructions in an AI calculation, each instruction does not need high precision and there is a lot of redundant precision that can be excluded
- Can use some form of estimation without sacrificing accuracy
- 16 and 98 bit precision is industry accepted
- Full-precision computing: Model weights are larger so more storage needed
- With "smaller" computations like a*b, the answer will always be the same regardless if its 32 or 64 bits
- In a neural network, if one neuron makes a mistakes then the other neuron can capture the sginal
    - Meaning: the neural network will be given the one big task. Each neuron will do little bits of the task. If one neuron messes up their tasks, then the other neurons will still be accurate enough to give a reasonable prediction

Time: XX

Title:

Objective:

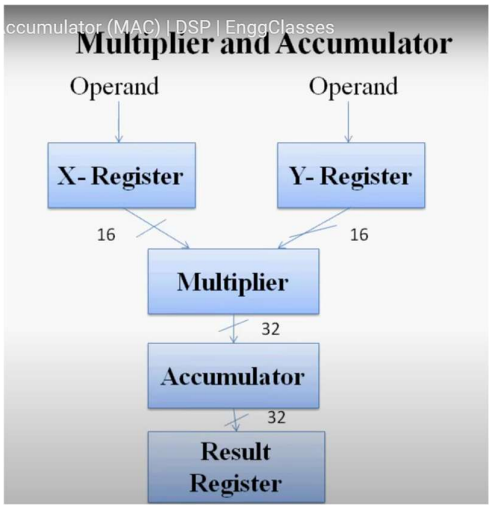
- **Point 1**

Weekly Logbook Entry Template

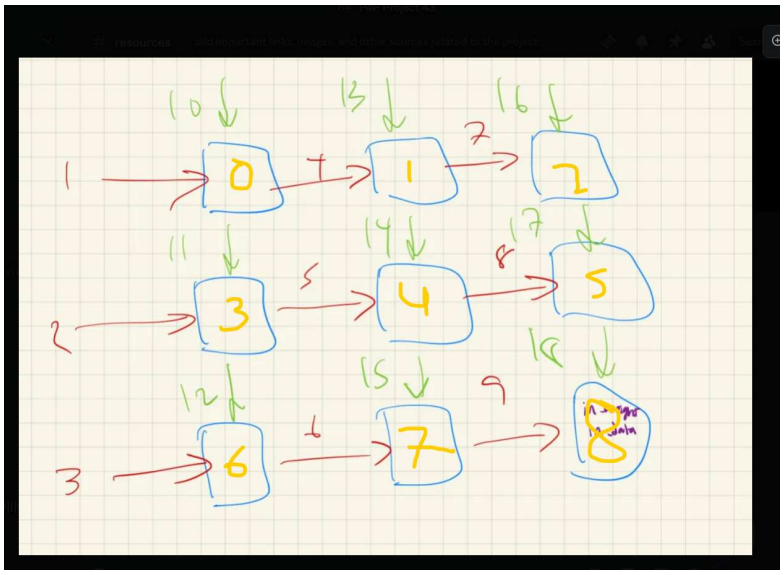
Week 10 19/05/25 – 22/05/25:

Time: XX

Objective: Creating a basic systolic array system in VHDL and test via ModelSim



- Basically, the result and the accumulation stays within the PE. It doesn't actually get passed through the systolic array. The output of the systolic array is the result register
- This is the systolic array architecture that we are trying to implement. Between each PE there is an interconnection signal that needs to be instantiated. The red lines represent the data value passing through the green represent the weight value passing through.



Systolic Array Implementation on VHDL:

- There is something wrong with the processing element where we are struggling to implement the accumulator register
- The systolic array itself is passing through the data properly when we only do multiplication
- The data is staggered through from the top left corner to the bottom right corner
- Manually injecting the data

Ideas to implement it better:

- Use a control unit to pass the data through the entry points of the systolic array
    - Consider the difference between row major vs column major
- Try adding the memory buffer of the PE in the top level of the systolic array

To do:

- Get a proper understanding on how systolic arrays work
- Then move onto the sparse matrices

Found a project from Coloumbia University that basically replicates what we're trying to do:
https://www.cs.columbia.edu/~sedwards/classes/2025/4840-spring/proposals/Systolic.pdf
https://www.cs.columbia.edu/~sedwards/classes/2025/4840-spring/designs/Systolic.pdf
https://www.cs.columbia.edu/~sedwards/classes/2025/4840-spring/index.html

Links that might be useful but have not been saved
https://www.cse.wustl.edu/~roger/560M.f17/01653825.pdf

XX X XXX 2025 **Time: XX**

Title:

How do Systolic Arrays work?
To implement systolic arrays in FPGA, we must understand the fundamental behaviour of a systolic
xiii
array. Systolic arrays are made up of a network of processing elements to perform simple operations alongside other PEs (JOHN P. CONDORODIS)
So, how does a processing element work?
To put simply, a processing element is just a cell that 'processes' something. For our case, we want
this to do a multiply and accumulate operation. For the sake of simplicity, I will just use multiply as
an example for my understanding.
"The processing element (PE) is essentially an arithmetic logic unit (ALU) with attached working
registers and local memory (Briggs and Hwang 1984)."
If we take this quote and apply to our PE, to find the input and outputs, we simply think of "What
do we need to give the ALU so that it performs multiplication?"
This tells us that we just need input_a and input_b with output_c

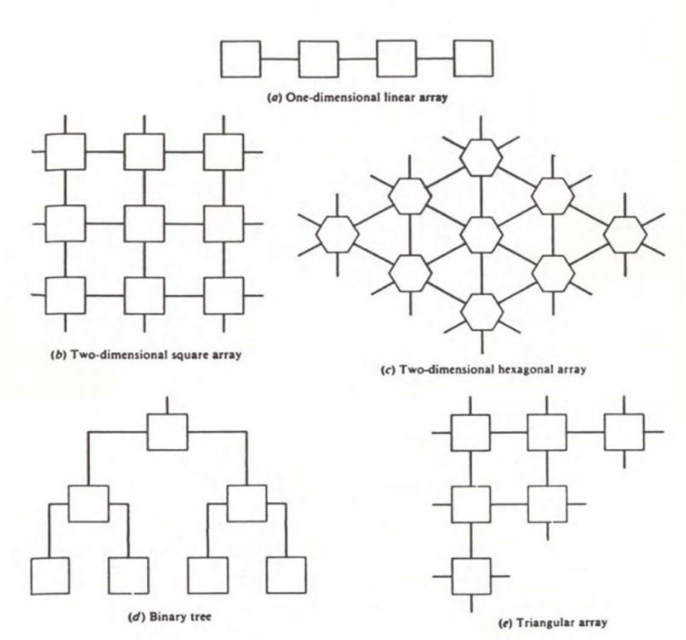


Figure 2. Typical Systolic Architecture Configurations.